

NOTICE

- Deadlines
 - (5/20 9:00 AM) PA3 Grades will be released
 - (6/01 8:00 AM) Midterm Quiz 4
 - 5/20: No class; lecture recordings will be made available asynchronously

RACE CONDITION OFTEN LEADS TO A SECURITY VULNERABILITY



DIRTY COW Vulnerability ([Video](#))

CS 374: OPERATING SYSTEMS I

PART IV – SYNCHRONIZATION II

Mon/Wed 12:00 – 1:50 PM

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI

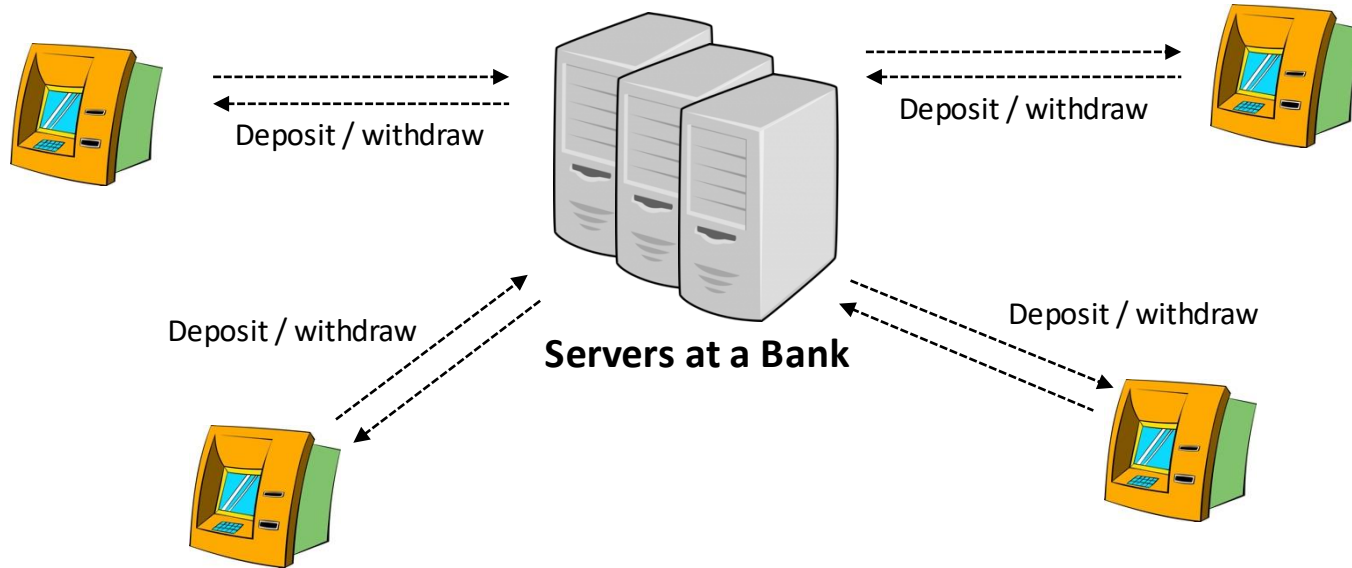
TOPICS FOR TODAY

- Part IV – Synchronization II
 - Recap:
 - Race condition + atomic operation
 - Mutex + critical section
 - Manage resources
 - Producer-consumer problem (Coke machine)
 - Deadlock
 - Provide abstraction & Offer standard interface
 - Semaphore
 - Semaphore in C

SYNCHRONIZATION

- **ATM Bank Server**

- The server(s) takes care of multiple deposit / withdrawal requests
- Bank want to make sure all the transactions are correct



ATM BANK SERVER V0.3

- **Multi-threaded**

- Receive a request
- Create a thread for processing it
- Process multiple request concurrently

```
void ProcessRequest(op, accountId, amount) {
    switch (op) {
        case OP_DEPOSIT:
            pthread_t *newTh = <mem alloc>;
            pthread_create(newTh, Deposit, info);
        case OP_WITHDRAW:
            pthread_t *newTh = <mem alloc>;
            pthread_create(newTh, Withdraw, info);
    }
}

void Deposit(accountId, amount) {
    account = GetAccount(accountId);
    account->balance += amount;
    StoreAccount(account);
}

int main(void) {
    int op = -1;
    int accountId, amount = -1, -1;

    while (1) {
        ReceiveRequest(&op, &accountId, &amount);
        ProcessRequest(op, accountId, amount);
    }

    return 0;    // code only reaches here if the server terminates
}
```

PROBLEM OF MULTI-THREADED BANK SERVER

- **OS controls the concurrency**
 - Can't control context switch
 - Can't control the execution order
 - Can't guarantee the balance

Thread A: Deposit \$200

1. LOAD r1 = \$400
3. ADD r1 \$200 (\$600)
4. STORE r1 (\$600)

Thread B: Deposit \$100

2. LOAD r1 = \$400
5. ADD r1 \$100 (\$500)
6. STORE r1 (\$500)

Final Balance: \$500?!

```
void ProcessRequest(op, accountId, amount) {  
    switch (op) {  
        case OP_DEPOSIT:  
            pthread_t *newTh = <mem alloc>;  
            pthread_create(newTh, Deposit, info);  
        case OP_WITHDRAW:  
            pthread_t *newTh = <mem alloc>;  
            pthread_create(newTh, Withdraw, info);  
    }  
}
```

```
void Deposit(accountId, amount) {  
    account = GetAccount(accountId);  
    account->balance += amount;  
    StoreAccount(account);  
}
```

```
int main(void) {  
    int op = -1;  
    int accountId, amount = -1, -1;  
  
    while (1) {  
        ReceiveRequest(&op, &accountId, &amount);  
        ProcessRequest(op, accountId, amount);  
    }  
}
```

```
return 0;    // code only reaches here if the server terminates  
}
```

PROBLEM OF MULTI-THREADED BANK SERVER

- **Race condition**

- **Definition:** an undesirable scenario in which
 - 2+ threads access a shared resource concurrently, *and*
 - The result depends on the order of execution



Now Final Balance: \$600?!

PROBLEM OF MULTI-THREADED BANK SERVER

- **Race condition**

- **Definition:** an undesirable scenario in which
 - 2+ threads access a shared resource concurrently, *and*
 - The result depends on the order of execution
- **Race condition happens**
 - Threads are running concurrently
 - Those threads access to the same shared resource
 - At least one of these threads “writes” to the resource

ATOMIC OPERATION

• Problem

- Deposit() is *indivisible* ←
- A ctx switch can happen in the middle
- Make sure to execute it at once

```
void ProcessRequest(op, accountId, amount) {
    switch (op) {
        case OP_DEPOSIT:
            pthread_t *newTh = <mem alloc>;
            pthread_create(newTh, Deposit, info);
        case OP_WITHDRAW:
            pthread_t *newTh = <mem alloc>;
            pthread_create(newTh, Withdraw, info);
    }
}

void Deposit(accountId, amount) {
    account = GetAccount(accountId);
    account->balance += amount;
    StoreAccount(account);
}

int main(void) {
    int op = -1;
    int accountId, amount = -1, -1;

    while (1) {
        ReceiveRequest(&op, &accountId, &amount);
        ProcessRequest(op, accountId, amount);
    }

    return 0;    // code only reaches here if the server terminates
}
```

ATOMIC OPERATION

- **Problem**

- Deposit() is *indivisible*
- A ctx switch can happen in the middle
- Make sure to execute it at once

- **Atomic operation**

- Code that must be run w/o interrupt
- Either completes fully or no execution

```
void ProcessRequest(op, accountId, amount) {  
    switch (op) {  
        case OP_DEPOSIT:  
            pthread_t *newTh = <mem alloc>;  
            pthread_create(newTh, Deposit, info);  
        case OP_WITHDRAW:  
            pthread_t *newTh = <mem alloc>;  
            pthread_create(newTh, Withdraw, info);  
    }  
}
```

```
{  
    void Deposit(accountId, amount) {  
        account = GetAccount(accountId);  
        account->balance += amount;  
        StoreAccount(account);  
    }  
}
```

```
int main(void) {  
    int op = -1;  
    int accountId, amount = -1, -1;  
  
    while (1) {  
        ReceiveRequest(&op, &accountId, &amount);  
        ProcessRequest(op, accountId, amount);  
    }  
  
    return 0;    // code only reaches here if the server terminates  
}
```

ATOMIC OPERATION – TWO APPROACHES

- **Hardware-level atomic operations**

- x86 instruction: lock add [balance], amount
- This only works for a simple operation (e.g., with single variable)

- **Software-level atomic operations**

- Use a mutex for a complex code blocks (next slides)
- Mutex works for multiple lines of code

MUTUAL EXCLUSION (MUTEX)

- Mutex

- **Definition:** a lock ensures that only one thread can run a critical section at a time
- **Three operations**
 - **Lock** before running a critical section
 - **Unlock** after running the critical section
 - **Wait** while someone locked the code

```
pthread_mutex_t deposit_lock;
```

```
void ProcessRequest(op, accountId, amount) {  
    switch (op) {  
        case OP_DEPOSIT:  
            ...  
    }  
}
```

```
void Deposit(accountId, amount) {  
    pthread_mutex_lock(&foo_mutex);    // lock before the atomic op.  
    account = GetAccount(accountId);  
    account->balance += amount;  
    StoreAccount(account);  
    pthread_mutex_unlock(&foo_mutex);  // unlock after the atomic op.  
}
```

```
int main(void) {  
    int op = -1;  
    int accountId, amount = -1, -1;  
    pthread_mutex_init(&deposit_lock, NULL);  
  
    while (1) {  
        ReceiveRequest(&op, &accountId, &amount);  
        ProcessRequest(op, accountId, amount);  
    }  
}
```

```
return 0;    // code only reaches here if the server terminates
```

MUTUAL EXCLUSION (MUTEX)

- Mutex
 - **Definition:** a lock ensures that only one thread can run a critical section at a time
- Critical section 
 - **Definition:** the code region protected by mutex lock and unlock (only one thread can be inside this)

```
pthread_mutex_t deposit_lock;

void ProcessRequest(op, accountId, amount) {
    switch (op) {
        case OP_DEPOSIT:
            ...
    }
}

void Deposit(accountId, amount) {
    pthread_mutex_lock(&foo_mutex); // lock before the atomic op.
    account = GetAccount(accountId);
    account->balance += amount;
    StoreAccount(account);
    pthread_mutex_unlock(&foo_mutex); // unlock after the atomic op.
}

int main(void) {
    int op = -1;
    int accountId, amount = -1, -1;
    pthread_mutex_init(&deposit_lock, NULL);

    while (1) {
        ReceiveRequest(&op, &accountId, &amount);
        ProcessRequest(op, accountId, amount);
    }

    return 0; // code only reaches here if the server terminates
}
```

MUTUAL EXCLUSION (MUTEX)

- Mutex
 - **Definition:** a lock ensures that only one thread can run a critical section at a time
 - **Program operations**
 - **Lock:** other threads should wait until the one holding the lock finishes
 - **Unlock:** wakes up any waiting threads

```
pthread_mutex_t deposit_lock;

void ProcessRequest(op, accountId, amount) {
    switch (op) {
        case OP_DEPOSIT:
            ...
    }
}

void Deposit(accountId, amount) {
    pthread_mutex_lock(&foo_mutex);    // lock before the atomic op.
    account = GetAccount(accountId);
    account->balance += amount;
    StoreAccount(account);
    pthread_mutex_unlock(&foo_mutex);  // unlock after the atomic op.
}

int main(void) {
    int op = -1;
    int accountId, amount = -1, -1;
    pthread_mutex_init(&deposit_lock, NULL);

    while (1) {
        ReceiveRequest(&op, &accountId, &amount);
        ProcessRequest(op, accountId, amount);
    }

    return 0;    // code only reaches here if the server terminates
}
```

MUTUAL EXCLUSION (MUTEX)

- Common mutex mistakes
 - **Mistake 1:** hold the lock too long
 - `pthread_mutex_lock(&lock);`
 - `sleep(10000000);`
 - `pthread_mutex_unlock(&lock);`
 - **Mistake 2:** forgot to unlock
 - `pthread_mutex_lock(&lock);`
 - `if (error) return;`
 - `pthread_mutex_unlock(&lock);`
 - **Mistake 3:** use the incorrect lock
 - `pthread_mutex_lock(&lock_A);`
 - `account->balance += amount;`
 - `pthread_mutex_unlock(&lock_B);`

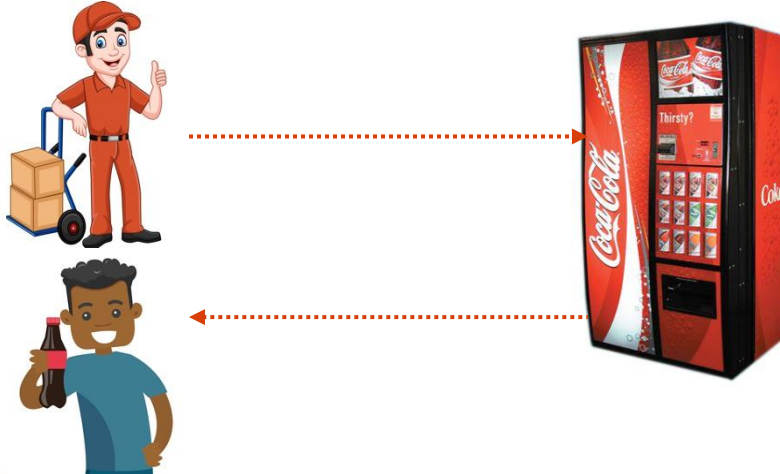
TOPICS FOR TODAY

- Part IV – Synchronization II
 - Recap:
 - Race condition + atomic operation
 - Mutex + critical section
 - Manage resources
 - Producer-consumer problem (Coke machine)
 - Deadlock
 - Provide abstraction & Offer standard interface
 - Semaphore
 - Semaphore in C

PRODUCER-CONSUMER PROBLEM

- A coke machine

- Two workers (or threads):
 - Producer: fills the coke machine
 - Producer needs to wait if the machine is full
 - Consumer: takes cokes from the machine
 - Consumer needs to wait if it is empty



COKE MACHINE DRAFT

- **Naïve implementation in C**

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Consumer thread gets a coke

- **Problem I:**

- Producer puts cokes when it's full
- Consumer takes a coke when it's empty

```
#define MACHINE_CAPACITY    64
static struct coke_machine;

void producer_fn() {
    while (1) {
        enqueue(acoke, coke_machine);
    }
}

void consumer_fn() {
    while (1) {
        acoke = dequeue(coke_machine);
    }
}

int main(void) {
    pthread_t producer, consumer;

    pthread_create(&producer, NULL, producer_fn, NULL);
    pthread_create(&consumer, NULL, consumer_fn, NULL);

    pthread_join(producer, NULL);
    pthread_join(consumer, NULL);

    return 0;           // code only reaches here if the machine is broken
}
```

COKE MACHINE V0.1

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it's full
 - Consumer thread gets a coke
 - Consumer waits if it's empty

- Solution I:

- Busy-waiting (or spinning) ▲
- Repeatedly checks if a condition is true

```
#define MACHINE_CAPACITY    64
static struct coke_machine;

void producer_fn() {
    while (1) {
        while (machine == full) {};
        enqueue(acoke, coke_machine);
    }
}

void consumer_fn() {
    while (1) {
        while (machine == empty) {};
        acoke = dequeue(coke_machine);
    }
}

int main(void) {
    pthread_t producer, consumer;

    pthread_create(&producer, NULL, producer_fn, NULL);
    pthread_create(&consumer, NULL, consumer_fn, NULL);

    pthread_join(producer, NULL);
    pthread_join(consumer, NULL);

    return 0;           // code only reaches here if the machine is broken
}
```

COKE MACHINE V0.1

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- Problem II:

- Race condition can occur
 - Producer checks # cokes
 - Context switch!
 - Consumer takes a coke
 - Producer increases the coke # by 3

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {
```

```
    while (1) {
```

```
        while (machine == full) {};
```

```
        enqueue(acoke, coke_machine);
```

```
    }
```

```
}
```

```
void consumer_fn() {
```

```
    while (1) {
```

```
        while (machine == empty) {};
```

```
        acoke = dequeue(coke_machine);
```

```
    }
```

```
}
```

```
int main(void) {
```

```
    pthread_t producer, consumer;
```

```
    pthread_create(&producer, NULL, producer_fn, NULL);
```

```
    pthread_create(&consumer, NULL, consumer_fn, NULL);
```

```
    pthread_join(producer, NULL);
```

```
    pthread_join(consumer, NULL);
```

```
    return 0;
```

```
    // code only reaches here if the machine is broken
```

```
}
```

```
ncoke = count(coke_machine)
ncoke += 1
coke_machine->cokes = ncoke
```

```
ncoke = count(coke_machine)
ncoke -= 1
coke_machine->cokes = ncoke
```

COKE MACHINE V0.2

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- Solution II:

- **Mutex**
- *Lock* coke machine before en-/de-queuing
- *Unlock* coke machine after en-/de-queuing

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == full) {};  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == empty) {};  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;
```

```
....
```

```
    return 0; // code only reaches here if the machine is broken  
}
```

COKE MACHINE V0.2

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- Problem III:

- **Deadlock** can occur
 - Producer enqueue
 - It locks and wait for an empty slot(s)
 - Consumer waits “unlock”

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == full) {};  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        pthread_mutex_lock(&machine);  
        while (machine == empty) {};  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;  
  
    ....
```

```
    return 0; // code only reaches here if the machine is broken  
}
```

COKE MACHINE V0.2

- **Deadlock**

- **Definition:** a “state” where:

- 2+ threads are each waiting for a resource held by the other
 - none can make progress

- **Deadlock happens**

- **Mutex:** a resource can be used by only one thread at a time
 - **Hold and wait:** a thread hold the resource while waiting for another
 - **No preemption:** a resource cannot be forcibly taken by a thread
 - **Circular wait:** threads form a cycle, each waiting for the next thread’s resource

- **Solution:** break one of the four conditions above

COKE MACHINE V0.2

- **Deadlock**

- **Definition:** a “state” where:
 - 2+ threads are each waiting for a resource held by the other
 - none can make progress

- **Starvation**

- **Definition:** one specific thread keeps getting not scheduled (others proceed fine)

COKE MACHINE V0.3

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Solution III:**

- *Break hold and wait*
- No more deadlock

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        while (machine == full) {};  
        pthread_mutex_lock(&machine);  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        while (machine == empty) {};  
        pthread_mutex_lock(&machine);  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;
```

```
....
```

```
    return 0;
```

```
}
```

```
// code only reaches here if the machine is broken
```

TOPICS FOR TODAY

- Part IV – Synchronization II
 - Recap:
 - Race condition + atomic operation
 - Mutex + critical section
 - Manage resources
 - Producer-consumer problem (Coke machine)
 - Deadlock
 - Provide abstraction & Offer standard interface
 - Semaphore
 - Semaphore in C

COKE MACHINE V0.3

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- Problem IV:

- Busy-wait “consumes CPUs”
- Not sure when busy-wait threads will run
 - Multiple threads run on CPUs
 - Starvation

```
#define MACHINE_CAPACITY 64
```

```
static struct coke_machine;
```

```
void producer_fn() {  
    while (1) {  
        while (machine == full) {};  
        pthread_mutex_lock(&machine);  
        enqueue(acoke, coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        while (machine == empty) {};  
        pthread_mutex_lock(&machine);  
        acoke = dequeue(coke_machine);  
        pthread_mutex_unlock(&machine);  
    }  
}
```

```
int main(void) {  
    pthread_t producer, consumer;
```

```
....
```

```
    return 0; // code only reaches here if the machine is broken  
}
```

HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore

- **Definition:** a synchronization object with a counter and a waiting queue
- **TL; DR:** Mutex + Counter + Sleep/Wake mechanism

- Semaphore operations

- **P():** *decrease* the counter by 1; will *wait* until a semaphore becomes positive
- **V():** *increase* the counter by 1; will *wake up* any thread that waits by P()

P() or sem_wait():

sem.value -= 1

if sem.value < 0:

add thread to sem.queue

sleep() // remove

V() or sem_post():

sem.value += 1

if sem.value <= 0:

pick one thread from sem.queue

wake_up() // move to ready Q

BUSY-WAIT VS. SEMAPHORE

- Busy-wait
 - Thread checks the while condition
 - If the resource is not yet available
 - It checks the while condition again
 - ... (do this forever... which burns CPU time)
- Semaphore
 - Thread is put into sleep by `sem_wait()`
 - If the condition has not been met, *it sleeps and yields CPUs*
 - Another thread calls `sem_post()`
 - If the condition is met, it proceeds... (no resource starvation)

HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore POSIX API

- `sem_t` semaphore: semaphore object
- `int sem_init(sem_t *sem, int pshared, unsigned int value)`
 - `pshared`: 0 – shared between a process' threads | 1 – shared between processes
 - `value`: counter value

HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore POSIX API

- `sem_t` semaphore: semaphore object
- `int sem_init(sem_t *sem, int pshared, unsigned int value)`
 - `pshared`: 0 – shared between a process' threads | 1 – shared between processes
 - `value`: counter value
- `int sem_destroy(sem_t *sem)`
- `int sem_wait(sem_t *sem)` : P()
- `int sem_post(sem_t *sem)` : V()

HOW TO ADDRESS BUSY-WAITING PROBLEMS

- Semaphore POSIX API

- `sem_t` semaphore: semaphore object
- `int sem_init(sem_t *sem, int pshared, unsigned int value)`
 - `pshared`: 0 – shared between a process' threads | 1 – shared between processes
 - `value`: counter value
- `int sem_destroy(sem_t *sem)`
- `int sem_wait(sem_t *sem)` : P()
- `int sem_post(sem_t *sem)` : V()
- `int sem_trywait(sem_t *sem)` : Does not wait if no resource is available
- `int sem_getvalue(sem_t *sem, int *val)`

- Note

- Need to link `-lpthread` in compilation (e.g., `gcc -o main main.c -lpthread`)

BINARY VS. COUNTING SEMAPHORE

- **Binary semaphore**

- Semaphore with counter 1 (operates like mutex)
- But threads cannot own the lock
- *Not a mutex*; mutex owned by a thread

```
sem_t lock;  
sem_init(&lock, 0, 1)  
...  
sem_wait(&lock)  
(...critical section...)  
sem_post(&lock)
```

- **Counting semaphore**

- Semaphore with counter N
- Manages N resources at the same time

```
sem_t lock;  
sem_init(&lock, 0, 64)  
...  
sem_wait(&lock)  
(...critical section or not...)  
sem_post(&lock)
```

COKE MACHINE V0.4

- In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

- **Solution IV:**

- Counting semaphore

Initialize with the # resources
1) Empty slots := 64 (capacity)
2) Filled slots := 0 (empty at first)

```
sem_t slots_filled;  
sem_t slots_empty;  
  
void producer_fn() {  
    while (1) {  
        sem_wait(&slots_empty);  
  
        enqueue(acoke, coke_machine);  
  
        sem_post(&slots_filled);  
    }  
}  
  
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
  
        acoke = dequeue(coke_machine);  
  
        sem_post(&slots_empty);  
    }  
}  
  
int main(void) {  
    int ret;  
    ret = sem_init(&slots_empty, 0, 64);  
    ret = sem_init(&slots_filled, 0, 0);  
    ....  
}
```

It decreases “filled slot” by one

It increases “empty slot” by one,
and wakes up any thread (i.e.,
producer thread) by sending a
signal to that thread

COKE MACHINE V0.4 – CONT'D

• In C

- A coke machine (can hold 64 cokes)
- Two workers (or threads):
 - Producer thread puts cokes
 - Producer waits if it is full
 - Consumer thread gets a coke
 - Consumer waits if it is empty

• Solution IV:

- Counting semaphore
- Binary semaphore

Initialize with the # resources
1) Empty slots := 64 (capacity)
2) Filled slots := 0 (empty at first)
3) Bin. semaphore := 1

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;
```

```
void producer_fn() {  
    while (1) {  
        sem_wait(&slots_empty);  
        sem_wait(&mutex);  
        enqueue(acoke, coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_filled);  
    }  
}
```

The semaphore only allows one thread to enqueue (or dequeue)

```
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}
```

It decreases “filled slot” by one

It increases “empty slot” by one, and wakes up any thread (i.e., producer thread) by sending a signal to that thread

```
int main(void) {  
    int ret;  
    ret = sem_init(&slots_empty, 0, 64);  
    ret = sem_init(&slots_filled, 0, 0);  
    ret = sem_init(&mutex, 0, 1);  
    ....  
}
```

COKE MACHINE V0.4 – CONT'D

- **Scenario I: machine empty**
 - Consumer requests a coke
 - It decreases “slots_filled” by 1
 - “slots_filled” becomes -1
 - Consumer sleeps
 - Producer puts a coke
 - It decreases “slots_empty” by 1
 - It adds a coke to the machine
 - It signals the thread waiting by “slots_filled”
 - Consumer **wakes up** and run
 - It decreases “slot_filled” by 1

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;  
  
void producer_fn() {  
    while (1) {  
        sem_wait(&slots_empty);  
        sem_wait(&mutex);  
        enqueue(acoke, coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_filled);  
    }  
}  
  
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}  
  
int main(void) {  
    int ret;  
    ret = sem_init(&slots_empty, 0, 64);  
    ret = sem_init(&slots_filled, 0, 0);  
    ret = sem_init(&mutex, 0, 1);  
    ....  
}
```

COKE MACHINE V0.4 – CONT'D

- **Scenario II: machine full**

- Producer puts a coke
 - It decreases “slots_empty” by 1
 - “slots_empty” becomes -1
 - Producer sleeps
- Consumer requests a coke
 - It decreases “slots_filled” by 1
 - It gets a coke from the machine
 - It signals the thread waiting by “slots_empty”
- Producer **wakes up** and run
 - It decreases “slot_empty” by 1

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;  
  
void producer_fn() {  
    while (1) {  
        sem_wait(&slots_empty);  
        sem_wait(&mutex);  
        enqueue(acoke, coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_filled);  
    }  
}  
  
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}  
  
int main(void) {  
    int ret;  
    ret = sem_init(&slots_empty, 0, 64);  
    ret = sem_init(&slots_filled, 0, 0);  
    ret = sem_init(&mutex, 0, 1);  
    ....  
}
```

DEADLOCK POSSIBLE WITH SEMAPHORE

- Order matters
 - Switch `sem_wait()`s in Producer
 - Producer locks “mutex”
 - Producer waits for “empty slot”
 - Consumer can’t dequeue b/c of “mutex”
 - **Deadlock!**
 - Switch `sem_post()`s in Producer
 - Producer decreases “slots_filled”
 - Consumer wakes up
 - Consumer can’t dequeue
 - Producer unlocks the mutex
 - Runs... (no deadlock)

Care Must Be Taken!

```
sem_t mutex;  
sem_t slots_filled;  
sem_t slots_empty;
```

```
void producer_fn() {  
    while (1) {  
        sem_wait(&mutex);  
        sem_wait(&slots_empty);  
        enqueue(acoke, coke_machine);  
        sem_post(&slots_filled);  
        sem_post(&mutex);  
    }  
}
```

```
void consumer_fn() {  
    while (1) {  
        sem_wait(&slots_filled);  
        sem_wait(&mutex);  
        acoke = dequeue(coke_machine);  
        sem_post(&mutex);  
        sem_post(&slots_empty);  
    }  
}
```

```
int main(void) {  
    int ret;  
    ret = sem_init(&mutex, 0, 1);  
    ret = sem_init(&slots_empty, 0, MACHINE_CAPACITY);  
    ret = sem_init(&slots_filled, 0, 0);
```

```
..... // omit the pthread_create / _join  
}
```

TOPICS FOR TODAY

- Part IV – Synchronization II
 - Recap:
 - Race condition + atomic operation
 - Mutex + critical section
 - Manage resources
 - Producer-consumer problem (Coke machine)
 - Deadlock
 - Provide abstraction & Offer standard interface
 - Semaphore
 - Semaphore in C

Thank You!

Mon/Wed 12:00 – 1:50 PM

Sanghyun Hong

sanghyun.hong@oregonstate.edu



Oregon State
University



TRUE AI
Trustworthy and Responsible AI